

Car Rental - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Pick branch + car type -> system chooses a free car -> price it -> pay -> later return at drop branch.

Story - how this LLD works

You want an SUV from Branch B1 for the weekend. The system filters free SUVs, then a booking strategy picks which one (least used, or cheapest). Price can be by hours or by km. Payment can be card or wallet. Two people booking the last SUV at the same time? AtomicBoolean makes sure only one wins.

Who does what

BookingService: Main coordinator (Singleton). bookVehicle / returnVehicle.

Branch: A location that holds vehicles.

Vehicle (Sedan/SUV): The car. Has price, status, and a tiny lock flag isBooked.

BookingStrategy: Which car to pick from the free list.

PricingStrategy: How much to charge.

PaymentStrategy: How customer pays.

Booking: The order record linking user, car, times, branches, money.

AtomicBoolean: compareAndSet(False, True) = claim the car safely under concurrency.

Main pieces

BookingService

- strategies
- repos
- book / return

Branch

- vehicles by type

Booking

- user, car
- money, status

Vehicle + lock

- isBooked CAS
- bookingCount

Step-by-step at runtime

1. bookVehicle(branch, SUV, start, end, user, payment)...
2. Load branch inventory; keep only AVAILABLE and not already flagged booked.
3. BookingStrategy tries cars; first successful compareAndSet wins the car.
4. PricingStrategy calculates amount for the time/distance.
5. Create Booking (CREATED). Try payment.
6. Payment OK -> CONFIRMED, car status BOOKED, save booking.
7. Payment fail -> release isBooked flag so someone else can take the car.
8. returnVehicle: mark COMPLETED, clear isBooked, add car to drop branch.

Why this design (plain words)

Why 3 Strategies? - Choosing a car, pricing, and paying change for different business rules - separately.

Why Singleton service? - One booking desk for the app (demo). Note: first init locks strategies.

Why AtomicBoolean? - Same idea as parking spot lock - race for last car.

Remember

- Time-slot overlap checker exists in utils but is NOT wired - currently one booking per car via flag.
- Payment strategy is passed per booking call; booking/pricing strategies are fixed on singleton.
- Demo: two threads fight for one SUV - only one booking succeeds.

OOP in this project

Encapsulation - Lock flag hidden inside AtomicBoolean.

Abstraction - Strategies only expose one method each.

Inheritance - Sedan/SUV extend Vehicle.

Polymorphism - Service calls bookVehicle/calculatePrice without knowing concrete strategy.

DESIGN.md - full file

Complete markdown from lld/car_rental/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Car Rental System - LLD Q&A

Run: `python -m lld.car_rental.main`

Patterns: Strategy · Singleton · Factory · Repository · Thread safety

Q1. What problem are we solving?

A. Design a multi-branch car rental system where users can:

Book a vehicle (Sedan / SUV) from a branch for a time window

Pick up at one branch and drop at another

Get price via a pricing strategy (hourly or distance-based)

Have a vehicle chosen via a booking strategy (least-booked or cheapest)

Pay via payment strategy (credit card or wallet)

Return the vehicle to the drop branch

Handle concurrent booking attempts safely

Q2. What are the core entities?

- Entity - Responsibility
- Vehicle (Sedan, SUV) - Plate, rates, status, booking count, atomic is_booked
- Branch - Inventory of vehicles by type
- Booking - User, vehicle, pickup/drop branches, times, amount, statuses
- User - Renter identity
- BookingService - Singleton orchestrator for book + return
- AtomicBoolean - Thread-safe compare-and-set lock on a vehicle

Q3. How is the package structured?

```
car_rental/
|-- model/      # Vehicle, Branch, Booking, User, AtomicBoolean
|-- service/    # BookingService, PaymentProcessor
|-- strategy/   # booking/ · pricing/ · payment/
|-- factory/    # VehicleFactory
|-- repository/ # BranchRepository, BookingRepository
|-- enums/     # VehicleType, VehicleStatus, BookingStatus, ...
|-- utils/     # Date parsing, availability checker
```

Q4. Why three Strategy families?

- Strategy - Question it answers
- BookingStrategy - *Which* vehicle to pick? (least booked / cheapest)
- PricingStrategy - *How much* to charge? (hourly / distance)
- PaymentStrategy - *How* to pay? (card / wallet)
- Each varies independently - Open/Closed Principle without changing BookingService core logic.

Q5. Why Singleton for `BookingService`?

A. One booking orchestrator shared by the app, with injected repos and strategies. Uses double-checked locking for thread-safe lazy init.

Caveat: First `get_instance(...)` locks strategies forever; later calls with different strategies are ignored.

Q6. What is the booking flow?

```
BookingService.book_vehicle(branch_id, type, start, end, user, payment, ...)
-> get branch, filter AVAILABLE + not is_booked
-> booking_strategy.book_vehicle(vehicles)
    -> compare_and_set(False, True) # atomic claim
-> pricing_strategy.calculate_price(...)
-> create Booking (CREATED, PENDING)
-> PaymentProcessor.pay(...)
-> success: CONFIRMED, vehicle BOOKED, persist
-> failure: release is_booked, return None
```

Q7. What is the return flow?

```
return_vehicle(booking_id)
-> booking must be CONFIRMED
-> status = COMPLETED
-> is_booked.set(False)
-> add vehicle to drop_branch inventory
```

Q8. How is concurrency handled?

A. `AtomicBoolean.compare_and_set(False, True)` on each vehicle. Two threads booking the only SUV -> exactly one wins; the other gets "No vehicle could be booked."

Q9. How do you extend this design?

- Add... - How
- New vehicle type - Enum + subclass + VehicleFactory
- New booking algo - Implement BookingStrategy, inject at init
- New pricing - Implement PricingStrategy
- New payment - Implement PaymentStrategy, pass per booking
- Time-slot overlap - Wire VehicleAvailabilityChecker into booking

Q10. Common interview follow-ups

Q. Is time-slot availability checked?

VehicleAvailabilityChecker exists but is not wired into booking. Currently one booking per vehicle via is_booked, not per calendar slot.

Q. Payment vs booking/pricing injectability?

Payment is per-call; booking/pricing are fixed on the singleton - asymmetric by design in this demo.

Q. VehicleStatus.BOOKED vs is_booked?

Two parallel flags. Return resets is_booked but does not set status back to AVAILABLE - a known simplification/gotcha.

Source code - lld/car_rental/

29 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (86 lines)

```

1| import threading
2|
3| from .enums.vehicle_type import VehicleType
4| from .factory.vehicle_factory import VehicleFactory
5| from .model.branch import Branch
6| from .model.user import User
7| from .repository.booking_repository import BookingRepository
8| from .repository.branch_repository import BranchRepository
9| from .service.booking_service import BookingService
10| from .strategy.booking.least_booked_vehicle_strategy import LeastBookedVehicleStrategy
11| from .strategy.payment.credit_card_payment_strategy import CreditCardPaymentStrategy
12| from .strategy.payment.wallet_payment_strategy import WalletPaymentStrategy
13| from .strategy.pricing.hourly_pricing_strategy import HourlyPricingStrategy
14| from .utils.date_time_parser import DateTimeParser
15|
16|
17| def main() -> None:
18|     branch_repo = BranchRepository()
19|     booking_repo = BookingRepository()
20|
21|     branch1 = Branch("B1", "New York")
22|     branch2 = Branch("B2", "Boston")
23|     branch_repo.add_branch(branch1)
24|     branch_repo.add_branch(branch2)
25|
26|     branch1.add_vehicle(VehicleFactory.create(VehicleType.SEDAN, "NY1234", 25, 3.5))
27|     branch1.add_vehicle(VehicleFactory.create(VehicleType.SEDAN, "NY5678", 22, 3))
28|     branch1.add_vehicle(VehicleFactory.create(VehicleType.SUV, "NYB100", 30, 4))
29|
30|     branch2.add_vehicle(VehicleFactory.create(VehicleType.SEDAN, "BO1234", 25, 4))
31|
32|     user = User("U1", "John Doe", "john@example.com")
33|
34|     start = DateTimeParser.parse("21 May 7:30 AM 2025")
35|     end = DateTimeParser.parse("21 May 12:30 PM 2025")
36|
37|     booking_service = BookingService.get_instance(
38|         branch_repo,
39|         booking_repo,
40|         LeastBookedVehicleStrategy(),
41|         HourlyPricingStrategy(),
42|     )
43|
44|     print("-----")
45|
46|     def book_with_credit_card() -> None:
47|         print(f"{threading.current_thread().name} started!")
48|         booking_service.book_vehicle(
49|             "B1",
50|             VehicleType.SUV,
51|             start,
52|             end,
53|             user,
54|             CreditCardPaymentStrategy(),
55|             branch1,
56|             branch2,
57|             100.0,
58|         )
59|         print(f"{threading.current_thread().name} ended!")
60|
61|     def book_with_wallet() -> None:
62|         print(f"{threading.current_thread().name} started!")
63|         booking_service.book_vehicle(
64|             "B1",
65|             VehicleType.SUV,
66|             start,
67|             end,
68|             user,
69|             WalletPaymentStrategy(),
70|             branch1,
71|             branch2,
72|             100.0,
73|         )
74|         print(f"{threading.current_thread().name} ended!")
75|
76|     t1 = threading.Thread(target=book_with_credit_card)
77|     t2 = threading.Thread(target=book_with_wallet)
78|
79|     t1.start()
80|     t2.start()
81|
82|     print("-----")
83|
84|
85| if __name__ == "__main__":
86|     main()

```

FILE: service/booking_service.py (133 lines)

```

1| import threading
2| import uuid
3| from datetime import datetime
4|
5| from ..enums.booking_status import BookingStatus
6| from ..enums.vehicle_status import VehicleStatus
7| from ..enums.vehicle_type import VehicleType
8| from ..model.booking import Booking
9| from ..model.branch import Branch
10| from ..model.user import User
11| from ..repository.booking_repository import BookingRepository
12| from ..repository.branch_repository import BranchRepository
13| from ..strategy.booking.booking_strategy import BookingStrategy
14| from ..strategy.payment.payment_strategy import PaymentStrategy
15| from ..strategy.pricing.pricing_strategy import PricingStrategy
16| from .payment_processor import PaymentProcessor
17|
18|
19| class BookingService:
20|     _instance: "BookingService | None" = None
21|     _lock = threading.Lock()
22|
23|     def __init__(
24|         self,
25|         branch_repo: BranchRepository,
26|         booking_repo: BookingRepository,
27|         booking_strategy: BookingStrategy,
28|         pricing_strategy: PricingStrategy,
29|     ) -> None:
30|         self._branch_repo = branch_repo
31|         self._booking_repo = booking_repo
32|         self.booking_strategy = booking_strategy
33|         self.pricing_strategy = pricing_strategy
34|
35|     @classmethod
36|     def get_instance(
37|         cls,
38|         branch_repo: BranchRepository,
39|         booking_repo: BookingRepository,
40|         booking_strategy: BookingStrategy,
41|         pricing_strategy: PricingStrategy,
42|     ) -> "BookingService":
43|         if cls._instance is None:
44|             with cls._lock:
45|                 if cls._instance is None:
46|                     cls._instance = cls(
47|                         branch_repo,
48|                         booking_repo,
49|                         booking_strategy,
50|                         pricing_strategy,
51|                     )
52|         return cls._instance
53|
54|     def book_vehicle(
55|         self,
56|         branch_id: str,
57|         vehicle_type: VehicleType,
58|         start: datetime,
59|         end: datetime,
60|         user: User,
61|         payment_strategy: PaymentStrategy,
62|         pick_up_branch: Branch,
63|         drop_branch: Branch,
64|         distance_km: float,
65|     ) -> Booking | None:
66|         branch = self._branch_repo.get_branch(branch_id)
67|         if branch is None:
68|             print("Branch not found")
69|             return None
70|
71|         active_vehicles = [
72|             vehicle
73|             for vehicle in branch.get_vehicles_by_type(vehicle_type)
74|             if vehicle.status == VehicleStatus.AVAILABLE
75|             and not vehicle.is_booked.get()
76|         ]
77|
78|         if not active_vehicles:
79|             print(f"No active {vehicle_type.name} vehicles available.")
80|             return None
81|
82|         vehicle = self.booking_strategy.book_vehicle(active_vehicles)
83|
84|         if vehicle is None:
85|             print("No vehicle could be booked.")
86|             return None
87|
88|         amount = self.pricing_strategy.calculate_price(vehicle, start, end, distance_km)
89|
90|         booking = Booking(

```

(cont.) service/booking_service.py

```

91|         booking_id=str(uuid.uuid4()),
92|         user=user,
93|         vehicle=vehicle,
94|         pickup_branch=pick_up_branch,
95|         drop_branch=drop_branch,
96|         start_time=start,
97|         end_time=end,
98|         amount=amount,
99|     )
100|
101|     processor = PaymentProcessor(payment_strategy)
102|     if not processor.pay(booking):
103|         print("Payment failed")
104|         vehicle.is_booked.set(False)
105|         return None
106|
107|     booking.status = BookingStatus.CONFIRMED
108|     self._booking_repo.add_booking(booking)
109|
110|     vehicle.increment_booking_count()
111|     vehicle.status = VehicleStatus.BOOKED
112|
113|     print(booking)
114|
115|     return booking
116|
117| def return_vehicle(self, booking_id: str) -> None:
118|     booking = self._booking_repo.get_booking_by_id(booking_id)
119|     if booking is None:
120|         raise RuntimeError("Booking not found")
121|
122|     if booking.status != BookingStatus.CONFIRMED:
123|         raise RuntimeError("Vehicle is not currently booked")
124|
125|     booking.status = BookingStatus.COMPLETED
126|     booking.vehicle.is_booked.set(False)
127|
128|     drop_branch = booking.drop_branch
129|     drop_branch.add_vehicle(booking.vehicle)
130|     print(
131|         f"Vehicle returned to branch {drop_branch.city}: "
132|         f"{booking.vehicle.license_plate}"
133|     )

```

FILE: service/payment_processor.py (19 lines)

```

1| from ..enums.payment_status import PaymentStatus
2| from ..model.booking import Booking
3| from ..strategy.payment.payment_strategy import PaymentStrategy
4|
5|
6| class PaymentProcessor:
7|     def __init__(self, payment_strategy: PaymentStrategy) -> None:
8|         self._payment_strategy = payment_strategy
9|
10|     def pay(self, booking: Booking) -> bool:
11|         success = self._payment_strategy.process_payment(booking)
12|
13|         if success:
14|             booking.payment_status = PaymentStatus.SUCCESS
15|         else:
16|             booking.payment_status = PaymentStatus.FAILED
17|             print("Payment Failed!")
18|
19|         return success

```

FILE: model/atomic_boolean.py (21 lines)

```

1| import threading
2|
3|
4| class AtomicBoolean:
5|     def __init__(self, initial: bool = False) -> None:
6|         self._value = initial
7|         self._lock = threading.Lock()
8|
9|     def get(self) -> bool:
10|         return self._value
11|
12|     def set(self, value: bool) -> None:
13|         with self._lock:
14|             self._value = value
15|
16|     def compare_and_set(self, expect: bool, update: bool) -> bool:
17|         with self._lock:
18|             if self._value == expect:
19|                 self._value = update
20|             return True
21|         return False

```

FILE: model/booking.py (42 lines)

(cont.) model/booking.py

```

1| from dataclasses import dataclass
2| from datetime import datetime
3|
4| from ..enums.booking_status import BookingStatus
5| from ..enums.payment_status import PaymentStatus
6| from .branch import Branch
7| from .user import User
8| from .vehicle import Vehicle
9|
10|
11| def _format_datetime(value: datetime) -> str:
12|     hour = value.hour % 12 or 12
13|     return f'{value.day} {value.strftime("%b")} {hour}:{value.strftime("%M %p %Y")}'
14|
15|
16| @dataclass
17| class Booking:
18|     booking_id: str
19|     user: User
20|     vehicle: Vehicle
21|     pickup_branch: Branch
22|     drop_branch: Branch
23|     start_time: datetime
24|     end_time: datetime
25|     status: BookingStatus = BookingStatus.CREATED
26|     payment_status: PaymentStatus = PaymentStatus.PENDING
27|     amount: float = 0.0
28|
29|     def __str__(self) -> str:
30|         return (
31|             "\n"
32|             f"Booking ID: {self.booking_id}\n"
33|             f"User: {self.user.name if self.user else 'N/A'}\n"
34|             f"Pickup Time: {_format_datetime(self.start_time)}\n"
35|             f"Drop Time: {_format_datetime(self.end_time)}\n"
36|             f"Pickup Location: {self.pickup_branch.city if self.pickup_branch else 'N/A'}\n"
37|             f"Drop Location: {self.drop_branch.city if self.drop_branch else 'N/A'}\n"
38|             f"Vehicle Type: {self.vehicle.type.name if self.vehicle else 'N/A'}\n"
39|             f"Vehicle Number Plate: {self.vehicle.license_plate if self.vehicle else 'N/A'}\n"
40|             f"Amount: ${self.amount:.2f}\n"
41|             f"Status: {self.status.name}\n"
42|         )

```

FILE: model/branch.py (22 lines)

```

1| from dataclasses import dataclass, field
2|
3| from ..enums.vehicle_type import VehicleType
4| from .vehicle import Vehicle
5|
6|
7| @dataclass
8| class Branch:
9|     id: str
10|     city: str
11|     vehicles: dict[VehicleType, list[Vehicle]] = field(default_factory=dict)
12|
13|     def get_vehicles_by_type(self, vehicle_type: VehicleType) -> list[Vehicle]:
14|         return list(self.vehicles.get(vehicle_type, []))
15|
16|     def add_vehicle(self, vehicle: Vehicle) -> None:
17|         self.vehicles.setdefault(vehicle.type, []).append(vehicle)
18|
19|     def remove_vehicle(self, vehicle: Vehicle) -> None:
20|         vehicle_list = self.vehicles.get(vehicle.type)
21|         if vehicle_list is not None:
22|             vehicle_list.remove(vehicle)

```

FILE: model/sedan.py (7 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from .vehicle import Vehicle
3|
4|
5| class Sedan(Vehicle):
6|     def __init__(self, license_plate: str, price_per_hour: float, price_per_km: float) -> None:
7|         super().__init__(license_plate, price_per_hour, price_per_km, VehicleType.SEDAN)

```

FILE: model/suv.py (7 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from .vehicle import Vehicle
3|
4|
5| class SUV(Vehicle):
6|     def __init__(self, license_plate: str, price_per_hour: float, price_per_km: float) -> None:
7|         super().__init__(license_plate, price_per_hour, price_per_km, VehicleType.SUV)

```

FILE: model/user.py (8 lines)

```

1| from dataclasses import dataclass
2|
3|
4| @dataclass
5| class User:
6|     id: str
7|     name: str
8|     email: str

```

FILE: model/vehicle.py (23 lines)

```

1| from abc import ABC
2| from dataclasses import dataclass, field
3|
4| from ..enums.vehicle_status import VehicleStatus
5| from ..enums.vehicle_type import VehicleType
6| from ..atomic_boolean import AtomicBoolean
7|
8|
9| @dataclass
10| class Vehicle(ABC):
11|     license_plate: str
12|     price_per_hour: float
13|     price_per_km: float
14|     type: VehicleType
15|     status: VehicleStatus = VehicleStatus.AVAILABLE
16|     booking_count: int = 0
17|     is_booked: AtomicBoolean = field(default_factory=AtomicBoolean, init=False, repr=False)
18|
19|     def __post_init__(self) -> None:
20|         self.is_booked = AtomicBoolean(False)
21|
22|     def increment_booking_count(self) -> None:
23|         self.booking_count += 1

```

FILE: strategy/booking/booking_strategy.py (9 lines)

```

1| from abc import ABC, abstractmethod
2|
3| from ...model.vehicle import Vehicle
4|
5|
6| class BookingStrategy(ABC):
7|     @abstractmethod
8|     def book_vehicle(self, vehicles: list[Vehicle]) -> Vehicle | None:
9|         pass

```

FILE: strategy/booking/cheapest_booking_strategy.py (24 lines)

```

1| from ...enums.pricing_strategy_type import PricingStrategyType
2| from ...model.vehicle import Vehicle
3| from .booking_strategy import BookingStrategy
4|
5|
6| class CheapestBookingStrategy(BookingStrategy):
7|     def __init__(self, pricing_type: PricingStrategyType) -> None:
8|         self._pricing_type = pricing_type
9|
10|     def book_vehicle(self, vehicles: list[Vehicle]) -> Vehicle | None:
11|         sorted_vehicles = sorted(
12|             vehicles,
13|             key=lambda vehicle: (
14|                 vehicle.price_per_hour
15|                 if self._pricing_type == PricingStrategyType.TIME_BASED
16|                 else vehicle.price_per_km
17|             ),
18|         )
19|
20|         for vehicle in sorted_vehicles:
21|             if vehicle.is_booked.compare_and_set(False, True):
22|                 return vehicle
23|
24|         return None

```

FILE: strategy/booking/least_booked_vehicle_strategy.py (13 lines)

```

1| from ...model.vehicle import Vehicle
2| from .booking_strategy import BookingStrategy
3|
4|
5| class LeastBookedVehicleStrategy(BookingStrategy):
6|     def book_vehicle(self, vehicles: list[Vehicle]) -> Vehicle | None:
7|         sorted_vehicles = sorted(vehicles, key=lambda vehicle: vehicle.booking_count)
8|
9|         for vehicle in sorted_vehicles:
10|             if vehicle.is_booked.compare_and_set(False, True):
11|                 return vehicle
12|
13|         return None

```

FILE: strategy/payment/credit_card_payment_strategy.py (8 lines)

```

1| from ...model.booking import Booking
2| from .payment_strategy import PaymentStrategy
3|
4|
5| class CreditCardPaymentStrategy(PaymentStrategy):
6|     def process_payment(self, booking: Booking) -> bool:
7|         print(f"Processing credit card payment for booking: {booking.booking_id}")
8|         return True

```

FILE: strategy/payment/payment_strategy.py (9 lines)

(cont.) strategy/payment/payment_strategy.py

```

1| from abc import ABC, abstractmethod
2|
3| from ...model.booking import Booking
4|
5|
6| class PaymentStrategy(ABC):
7|     @abstractmethod
8|     def process_payment(self, booking: Booking) -> bool:
9|         pass

```

FILE: strategy/payment/wallet_payment_strategy.py (8 lines)

```

1| from ...model.booking import Booking
2| from .payment_strategy import PaymentStrategy
3|
4|
5| class WalletPaymentStrategy(PaymentStrategy):
6|     def process_payment(self, booking: Booking) -> bool:
7|         print(f"Processing wallet payment for booking: {booking.booking_id}")
8|         return True

```

FILE: strategy/pricing/distance_based_pricing_strategy.py (15 lines)

```

1| from datetime import datetime
2|
3| from ...model.vehicle import Vehicle
4| from .pricing_strategy import PricingStrategy
5|
6|
7| class DistanceBasedPricingStrategy(PricingStrategy):
8|     def calculate_price(
9|         self,
10|         vehicle: Vehicle,
11|         start: datetime,
12|         end: datetime,
13|         distance_km: float,
14|     ) -> float:
15|         return distance_km * vehicle.price_per_km

```

FILE: strategy/pricing/hourly_pricing_strategy.py (18 lines)

```

1| import math
2| from datetime import datetime
3|
4| from ...model.vehicle import Vehicle
5| from .pricing_strategy import PricingStrategy
6|
7|
8| class HourlyPricingStrategy(PricingStrategy):
9|     def calculate_price(
10|         self,
11|         vehicle: Vehicle,
12|         start: datetime,
13|         end: datetime,
14|         distance_km: float,
15|     ) -> float:
16|         minutes = int((end - start).total_seconds() // 60)
17|         hours = math.ceil(minutes / 60.0)
18|         return vehicle.price_per_hour * hours

```

FILE: strategy/pricing/pricing_strategy.py (16 lines)

```

1| from abc import ABC, abstractmethod
2| from datetime import datetime
3|
4| from ...model.vehicle import Vehicle
5|
6|
7| class PricingStrategy(ABC):
8|     @abstractmethod
9|     def calculate_price(
10|         self,
11|         vehicle: Vehicle,
12|         start: datetime,
13|         end: datetime,
14|         distance_km: float,
15|     ) -> float:
16|         pass

```

FILE: factory/vehicle_factory.py (19 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from ..model.sedan import Sedan
3| from ..model.suv import SUV
4| from ..model.vehicle import Vehicle
5|
6|
7| class VehicleFactory:
8|     @staticmethod
9|     def create(
10|         vehicle_type: VehicleType,
11|         license_plate: str,
12|         price_per_hour: float,
13|         price_per_km: float,
14|     ) -> Vehicle:
15|         if vehicle_type == VehicleType.SEDAN:
16|             return Sedan(license_plate, price_per_hour, price_per_km)
17|         if vehicle_type == VehicleType.SUV:
18|             return SUV(license_plate, price_per_hour, price_per_km)
19|         raise ValueError(f"Unsupported vehicle type: {vehicle_type}")

```

FILE: repository/booking_repository.py (20 lines)

```
1| from ..model.booking import Booking
2|
3|
4| class BookingRepository:
5|     def __init__(self) -> None:
6|         self._bookings: dict[str, Booking] = {}
7|
8|     def add_booking(self, booking: Booking) -> None:
9|         self._bookings[booking.booking_id] = booking
10|
11|     def get_booking_by_id(self, booking_id: str) -> Booking | None:
12|         return self._bookings.get(booking_id)
13|
14|     def remove_booking(self, booking_id: str) -> None:
15|         booking = self._bookings.pop(booking_id, None)
16|         if booking is not None:
17|             booking.vehicle.is_booked.set(False)
18|
19|     def get_all_bookings(self) -> list[Booking]:
20|         return list(self._bookings.values())
```

FILE: repository/branch_repository.py (12 lines)

```
1| from ..model.branch import Branch
2|
3|
4| class BranchRepository:
5|     def __init__(self) -> None:
6|         self._branch_map: dict[str, Branch] = {}
7|
8|     def add_branch(self, branch: Branch) -> None:
9|         self._branch_map[branch.id] = branch
10|
11|     def get_branch(self, branch_id: str) -> Branch | None:
12|         return self._branch_map.get(branch_id)
```

FILE: enums/booking_status.py (9 lines)

```
1| from enum import Enum
2|
3|
4| class BookingStatus(Enum):
5|     CREATED = "CREATED"
6|     CONFIRMED = "CONFIRMED"
7|     COMPLETED = "COMPLETED"
8|     CANCELLED = "CANCELLED"
9|     FAILED = "FAILED"
```

FILE: enums/payment_status.py (7 lines)

```
1| from enum import Enum
2|
3|
4| class PaymentStatus(Enum):
5|     PENDING = "PENDING"
6|     SUCCESS = "SUCCESS"
7|     FAILED = "FAILED"
```

FILE: enums/pricing_strategy_type.py (6 lines)

```
1| from enum import Enum
2|
3|
4| class PricingStrategyType(Enum):
5|     TIME_BASED = "TIME_BASED"
6|     DISTANCE_BASED = "DISTANCE_BASED"
```

FILE: enums/vehicle_status.py (8 lines)

```
1| from enum import Enum
2|
3|
4| class VehicleStatus(Enum):
5|     AVAILABLE = "AVAILABLE"
6|     BOOKED = "BOOKED"
7|     IN_SERVICE = "IN_SERVICE"
8|     DECOMMISSIONED = "DECOMMISSIONED"
```

FILE: enums/vehicle_type.py (6 lines)

```
1| from enum import Enum
2|
3|
4| class VehicleType(Enum):
5|     SEDAN = "SEDAN"
6|     SUV = "SUV"
```

FILE: utils/date_time_parser.py (18 lines)

(cont.) utils/date_time_parser.py

```

1| import re
2| from datetime import datetime
3|
4|
5| class DateTimeParser:
6|     _PATTERN = re.compile(
7|         r"^(d{1,2}) ([A-Za-z]{3}) (\d{1,2}):(\d{2}) (AM|PM) (\d{4})$"
8|     )
9|
10|     @staticmethod
11|     def parse(value: str) -> datetime:
12|         match = DateTimeParser._PATTERN.match(value)
13|         if not match:
14|             raise ValueError(f"Unable to parse datetime: {value}")
15|
16|         day, month, hour, minute, meridiem, year = match.groups()
17|         normalized = f"{int(day):02d} {month} {int(hour):02d}:{minute} {meridiem} {year}"
18|         return datetime.strptime(normalized, "%d %b %I:%M %p %Y")

```

FILE: utils/vehicle_availability_checker.py (31 lines)

```

1| from datetime import datetime
2|
3| from ..enums.booking_status import BookingStatus
4| from ..model.booking import Booking
5|
6|
7| class VehicleAvailabilityChecker:
8|     @staticmethod
9|     def is_available(
10|         bookings: list[Booking],
11|         start: datetime,
12|         end: datetime,
13|     ) -> bool:
14|         active_bookings = sorted(
15|             [
16|                 booking
17|                 for booking in bookings
18|                 if booking.status in (BookingStatus.CREATED, BookingStatus.CONFIRMED)
19|             ],
20|             key=lambda booking: booking.start_time,
21|         )
22|
23|         for booking in active_bookings:
24|             existing_start = booking.start_time
25|             existing_end = booking.end_time
26|
27|             overlaps = not (end < existing_start or start > existing_end)
28|             if overlaps:
29|                 return False
30|
31|         return True

```